BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
HYDERABAD CAMPUS

CS F342 COMPUTER ARCHITECTURE
COURSE PROJECT REPORT

GROUP NO. 36
QUESTION SET - 11

NAME	ID. NO
SOHAM ERANDE	2022AAPS1376H
AMOGH G C	2022AAPS0254H
ACHYUT KHANDELWAL	2022A8PS1331H
HARIOM MAYURE	2022A3PS0534H

PROBLEM STATEMENT

Project
Set - 11

Statement: Design a 5-stage (as discussed in the lecture) pipeline RISC processor (with hazard detection and data forwarding unit as necessary) that can execute the following instructions:

0000 NAND reg1, reg2, reg3
0004 OR reg5, reg4, reg1
0008 LW reg7, 7(reg6)
0012 SUBI reg8, reg7, AA89
0016 ADD reg10, reg9, reg8

Initialize the register file with the following data
reg2 = 13578 reg3 = 234EE
reg4 = DD121 reg6 = 9
reg9 = CAF43

The instruction format is as follows:

31 28	27 24	23 20	19 16	15 0
OPCODE	Dest Reg (WN)	Source Reg 1 (RN1)	Source Reg 2 (RN2)	Immediate value

Opcode for each instruction:
Instruction 1: 0000
Instruction 2: 0001
Instruction 3: 0011
Instruction 4: 0111
Instruction 5: 1111

The processor has the following control signals:

- ALUSrc - Select the second input of ALU
- ALUOp (2 bits) - Control ALU operation
- MR - Read data from memory
- MW - Write data into memory
- MReg - Move data from memory to register
- EnIM - Read instruction memory contents
- EnRW - Write data into the register file
- FA - Forward A mux control (used in data forwarding circuitry)
- FB - Forward B mux control (used in data forwarding circuitry)
- IFIDWrite - Disable IF/ID change (used in hazard detection circuit)
- PCWrite - Disable PC change (used in hazard detection circuit)
- ST - Control signal of mux which changes all control signals to zero (used in hazard detection circuit)

Initialize PC with all zeros. The instruction memory is of size 32 bytes, and the processor has 16 registers, numbered from reg0 to reg15, and the registers are 32 bits wide. A read operation from the instruction memory outputs 4 consecutive bytes of information (starting from the byte address provided to the memory) at the positive edge of the clock if the EnIM control signal is high. Assume that the register file has two 32-bit read ports: RD1 and RD2, for data reading and one 32-bit write port: WD, for data writing. At the rising edge of the clock, the read ports RD1 and RD2 output the data from the registers whose addresses are available at RN1 and RN2, respectively. At the falling edge of a clock, data is written via the write port WD to the register file whose address is present at WN if the EnRW signal is true. Design the data memory size as per the requirement. All the data are in hexadecimal format unless specified.

Draw the detailed architecture-level diagram of the processor, depicting all the blocks (e.g., register file, instruction memory, ALU, PC, combinational functional blocks, hazard detection unit, forwarding unit, etc.). Describe each block individually, and create behavioral verilog models for each architectural block separately. Build the top-level structural model of the processor by instantiating and interconnecting the individual architectural blocks. Specify the size and format of all pipeline registers including the fields holding the decoded control signals as well as data. Show all the input, output, and control signal waveforms in the report.

1. PC REGISTER MODULE

```
`timescale 1ns / 1ps
module PC_Register(
    input PCWrite,clk, rst,
    input [31:0] PC_new,
    output reg [31:0]PC
    );
    initial begin
    PC = 0;
    end
    always@(posedge clk , posedge rst)
    begin
    if(PCWrite) begin
        if(rst)
        begin
            PC = 0;
        end
        else
            PC = PC_new;
        end
    end
endmodule
```

The Program Counter register holds the address of the next instruction to execute, ensuring sequential program flow. It takes control signals PCWrite, clk, and rst(reset), and a 32-bit input PC_new as the next instruction address. The PC is initialized to zero and updated on the rising edge of the clock or reset. The PC updates to PC_new when PCWrite is high, unless rst is also high — then it resets to zero. If PCWrite is low, the PC stays the same, allowing the processor to stall during hazards.

2. INSTRUCTION MEMORY

```
`timescale 1ns / 1ps
module Instruction_Memory(
    input EnIm,
    input[31:0]PC,
    output reg [31:0]MemData
    );
    reg [7:0]Mem[31:0];
    initial begin
    {Mem[0],Mem[1],Mem[2],Mem[3]} = 32'b00000001001000110000000000000000;
    {Mem[4],Mem[5],Mem[6],Mem[7]} = 32'b00010101010000010000000000000000;
    {Mem[8],Mem[9],Mem[10],Mem[11]}=32'b00110111011000000000000000000111;
    {Mem[12],Mem[13],Mem[14],Mem[15]}=32'b01111000011100001010101010001001;
    {Mem[16],Mem[17],Mem[18],Mem[19]}=32'b11111010100110000000000000000000;
    end
    always@(PC) begin
    if(EnIm)
    MemData = {Mem[PC], Mem[PC+1], Mem[PC+2], Mem[PC+3]};
    end
endmodule
```

Instruction Memory stores the program instructions. The inputs to this module is the 1-bit control signal EnIm and 32-bit PC. The EnIm signal indicates when the instruction memory should be accessed(for our problem set, it will always be 1). The PC gives us the address from where the instruction is to be fetched. Then the fetched instruction is given the name MemData and stored in the IFID pipeline register.

3. PC ADDER

```
`timescale 1ns / 1ps
module PC_Adder(
    input [31:0]PC,
    output [31:0]PC_new
    );
    assign PC_new = PC + 32'h4;
endmodule
```

This module takes the input as the current value of PC and updates it to PC+4 value for the next cycle. The output is taken as PC_new.

4. IFID PIPELINE REGISTER

```
`timescale 1ns / 1ps
module IFID(
    input clk,IFIDWrite,
    input [31:0]MemData,
    output reg [31:0]MemDataIFID
    );
    always@(posedge clk) begin
    if(IFIDWrite) begin
        MemDataIFID = MemData;
        end
    end
endmodule
```

This pipeline register is located just after the instruction memory. We just forward the contents of the instruction memory located at the address pointed by the PC. We are not sending the PC value here as there is no branch instruction in our dataset. Thus, the total size of this register is 32 bits only and it is updated at the positive edge of the clock as we want to retain the data for the entire clock cycle.

5. REGISTER FILE

```
`timescale 1ns / 1ps
module Register_File(
    input EnRWMEMWB,
    input[31:0]MemDataIFID,WriteData,
    input [3:0]WriteRegMEMWB,
    output reg [31:0]read_data1, read_data2,
    output reg [3:0]WriteReg,
    output reg [3:0]read_reg1, read_reg2
    );
    reg [31:0]regbank[15:0];
    initial begin
    regbank[0] = 0;
    regbank[1] = 32'h0;
    regbank[2] = 32'h13578;
    regbank[3] = 32'h234EE;
    regbank[4] = 32'hDD121;
    regbank[5] = 32'h0;
    regbank[6] = 32'h9;
    regbank[7] = 32'hab12459e;
    regbank[8] = 32'h0;
    regbank[9] = 32'hCAF43;
    regbank[10] = 32'h0;
    regbank[11] = 32'h0;
    regbank[12] = 32'h0;
    regbank[13] = 32'h0;
    regbank[14] = 32'h0;
    regbank[15] = 32'h0;
    end
    always@(*) begin
    read_reg1 = MemDataIFID[23:20];
    read_reg2 = MemDataIFID[19:16];
    WriteReg = MemDataIFID[27:24];
    read_data1 = regbank[read_reg1];
    read_data2 = regbank[read_reg2];
    if(EnRWMEMWB == 1)
        begin
        regbank[WriteRegMEMWB] = WriteData;
     end
     end
endmodule
```

This module stores all general-purpose registers and provides read and write access to them based on instruction operands. There are 15 32-bit registers available in the register bank. EnRWMEMWB(always 1 for our question set) control signal tells whether the data is to be written back to the register file or not. The 4-bit WriteRegMEMWB indicates in which register of the bank the data(WriteData) should be written. The MemDataIFID is the instruction which was stored in the IFID pipeline register. Its 24th to 27th bits give the destination register(Rd) address, 20th to 23th bits give the address of source register 1 and the 16th to 19th bits give the address of source register 2. The values coming out from the source registers are given

read_data1 and read_data2. The initial values of the register are given as per the question and other register values which are not mentioned in the question are initialized to 0.

6. CONTROL SIGNAL MODULE

```
`timescale 1ns / 1ps
module Control_Signal(
    input [31:0]MemDataIFID,
    output reg EnRW1, ALUSrc1, MR1, MW1, Mreg1,
    output reg [1:0] ALUOp1
    );
    wire [3:0]Opcode;
    assign Opcode = MemDataIFID[31:28];
        always@(Opcode) begin
        case(Opcode)
        4'b0000:
        begin
        EnRW1 = 1;
        ALUSrc1 = 0;
        MR1 = 0;
        MW1 = 0;
        Mreg1 = 0;
        ALUOp1 = 2'b00;
        end
        4'b0001:
        begin
        EnRW1 = 1;
        ALUSrc1 = 0;
        MR1 = 0;
        MW1 = 0;
        Mreg1 = 0;
        ALUOp1 = 2'b01;
        end
        4'b0011:
        begin
        EnRW1 = 1;
        ALUSrc1 = 1;
        MR1 = 1;
        MW1 = 0;
        Mreg1 = 1;
        ALUOp1 = 2'b11;
        end
        4'b0111:
        begin
        EnRW1 = 1;
        ALUSrc1 = 1;
        MR1 = 0;
        MW1 = 0;
        Mreg1 = 0;
        ALUOp1 = 2'b10;
        end
        4'b1111:
        begin
        EnRW1 = 1;
        ALUSrc1 = 0;
```

```
            MR1 = 0;
            MW1 = 0;
            Mreg1 = 0;
            ALUOp1 = 2'b11;
            end
            endcase
            end
endmodule
```

The Control Signal module generates the necessary control signals based on the instruction opcode to guide the operation of various processor components. It fetches the opcode from the 28th to 31th bits of the instruction coming from the IFID pipeline register(MemDataIFID). Our control signal module generates five 1-bit control signals: EnRW1(this indicates whether the data is to be written(1) in register or not(0)), ALUSrc1(it helps in recognizing when to use the immediate value(1) or when to use register data(0) with the help of ALUSrc_Mux), MR1(indicates when to fetch data from data memory), MW1(indicates when to write data in the data memory, which for our problem set is always 0), Mreg1(indicates when the data is to be moved from data memory to register). There is one 2-bit control signal also, which is ALUOp1(tells which operation is to be done by ALU).

7. CONTROL MUX

```
`timescale 1ns / 1ps
module Control_Mux(
    input EnRW1, ALUSrc1, MR1, MW1, Mreg1,
    input [1:0]ALUOp1,
    input ST,
    output EnRW, ALUSrc, MR, MW, Mreg,
    output [1:0]ALUOP
    );
    assign {EnRW, ALUSrc, MR, MW, Mreg,ALUOP} = (ST == 1 ?({EnRW1, ALUSrc1, MR1, MW1, Mreg1,ALUOp1}) : 7'b000000);
endmodule
```

The main function of this module is to detect memory hazard with the help of control signal ST and tell whether there is need of stalling(ST=0) or not(ST=1). The stalling is assured by assigning all the control signals equal to 0 because of which there is no read or write of the data is happening and everythings remains the same.

8. SIGN EXTEND MODULE

```
`timescale 1ns / 1ps
module Sign_Extend(
    input [31:0]MemDataIFID,
    output [31:0]out2
    );
    wire [15:0]Address;
    assign Address = MemDataIFID[15:0];
    assign out2 = (Address[15] == 0)? {16'b0,Address} : {16'b1111111111111111,Address};
endmodule
```

This module receives a 32-bit input from MemDataIFID. It extracts the lower 16 bits (bits 0 to 15) and examines the most significant bit (bit 15) of this 16-bit immediate value. If the most significant bit is 0, it performs zero extension by padding the upper 16 bits with zeros. If the most significant bit is 1, it performs sign extension by padding the upper 16 bits with ones. The output value is given name out2.

9. IDEX PIPELINE REGISTER

```
`timescale 1ns / 1ps
module IDEX(
    input [3:0]read_reg1, read_reg2,
    input clk,
    input [31:0] read_data1, read_data2, out2,
    input [3:0] WriteReg,
    input EnRW, ALUSrc, MR, MW, Mreg,
    input [1:0] ALUOp,
    output reg [31:0] read_data1IDEX, read_data2IDEX, out2IDEX,
    output reg [3:0] WriteRegIDEX,
    output reg EnRWIDEX, ALUSrcIDEX, MRIDEX, MWIDEX,MregIDEX,
    output reg [1:0] ALUOpIDEX,
    output reg [3:0]read_reg1IDEX, read_reg2IDEX
    );
    initial begin
    MRIDEX = 0;
    end
    always@(posedge clk) begin
    {read_data1IDEX, read_data2IDEX, out2IDEX} = {read_data1, read_data2, out2};
     WriteRegIDEX = WriteReg;
    read_reg1IDEX = read_reg1;
    read_reg2IDEX = read_reg2;
    {EnRWIDEX, ALUSrcIDEX, MRIDEX, MWIDEX, MregIDEX} = {EnRW, ALUSrc, MR, MW, Mreg};
    ALUOpIDEX = ALUOp;
    end
endmodule
```

This module is used to transfer data and control signals from the Instruction Decode (ID) stage to the Execution (EX) stage. All the input signals and data are latched and then forwarded to the execution cycle. At the positive edge of each clock cycle, the block updates the output registers with the corresponding input values. The size of this block for the given set of instructions is 115 bits.

10. ALU SOURCE MUX

```
`timescale 1ns / 1ps
module ALUSrc_Mux(
    input ALUSrcIDEX,
    input [31:0] read_data2IDEX, out2IDEX,
    output [31:0] ALUin
    );
    assign ALUin = (ALUSrcIDEX == 0)? read_data2IDEX : out2IDEX;
endmodule
```

This MUX is placed just before the MUX B, which helps in identifying whether the immediate data is to be taken or data from the register bank. If ALUSrcIDEX = 0, the register data is forwarded(read_data2IDEX), else the sign extended 32-bit value of the immediate value is to be forwarded(out2IDEX).

11. DATA FORWARDING UNIT MODULE

```
`timescale 1ns / 1ps
module Forwarding_Table(
    input [3:0]WriteRegEXMEM, WriteRegMEMWB,
    input [3:0]read_reg1IDEX, read_reg2IDEX,
    output reg [1:0] ForwardA, ForwardB
    );
    initial begin
    ForwardA = 2'b00;
    ForwardB = 2'b00;
    end
    always@(*) begin
    ForwardA = 2'b00;
    ForwardB = 2'b00;
    if(WriteRegEXMEM == read_reg1IDEX) begin
    ForwardA = 2'b10;
    end
    else if(WriteRegMEMWB == read_reg1IDEX) begin
    ForwardA = 2'b01;
    end
    if(WriteRegEXMEM == read_reg2IDEX) begin
    ForwardB = 2'b10;
    end
    else if(WriteRegMEMWB == read_reg2IDEX) begin
    ForwardB = 2'b01;
    end
    end
endmodule
```

This module detects whether there is need for data forwarding or not(due to data hazards) and then produces the corresponding control signals for MUX A and MUX B, which defines the data to be forwarded to the ALU. If there is no need for data forwarding, the value of control signal for the muxes is given 0. The need for data forwarding is checked by comparing the address of the register(at the EXMEM and MEMWB stages) where the data is to be written, with the address of the register at the IDEX stage, from where the data to be read. Control signal is given 2 when the data is to be forwarded from the EXMEM stage and is given 1 when the data is to be forwarded from the MEMWB stage.

12. ALU

```
`timescale 1ns / 1ps
module ALU(
    input [1:0] ALUOpIDEX,
    input [31:0] ALUin1, ALUin2,
    output reg [31:0] ALUOut
    );
    always@(*)
    begin
    casex(ALUOpIDEX)
    2'b00: ALUOut = ~(ALUin1 & ALUin2);
    2'b01: ALUOut = (ALUin1 | ALUin2);
    2'b10: ALUOut = (ALUin1 - ALUin2);
    2'b11: ALUOut = (ALUin1 + ALUin2);
    endcase
    end
endmodule
```

The ALU unit receives the control signal ALUOpIDEX which defines what operation is to be done by it. The inputs ALUin1 and ALUin2 are the outputs of the MUX A and MUX B respectively on which the operation is to be done. The operations on the inputs for the control signal ALUOpIDEX 00,01,10,11 respectively are NAND, OR, Subtraction and Addition operation. The output of ALU is given as ALUOut.

13. HAZARD DETECTION UNIT

```
`timescale 1ns / 1ps
module Hazard_detection(
    input MRIDEX,
    input [31:0]MemDataIFID,
    input [3:0] WriteRegIDEX,
    output reg IFIDWrite,
    output reg PCWrite, ST, EnIM
    );
    reg [3:0] rs1,rs2;
    initial begin
    EnIM = 1;
    end
    always@(*) begin
    rs1 = MemDataIFID[23:20];
    rs2 = MemDataIFID[19:16];
    IFIDWrite = (MRIDEX && ((WriteRegIDEX == rs1) || (WriteRegIDEX == rs2))) ? 0 : 1;
    PCWrite = (MRIDEX && ((WriteRegIDEX == rs1) || (WriteRegIDEX == rs2))) ? 0 : 1;
    ST = (MRIDEX & ((WriteRegIDEX == rs1) || (WriteRegIDEX == rs2))) ? 0 : 1;
    EnIM = (MRIDEX & ((WriteRegIDEX == rs1) || (WriteRegIDEX == rs2))) ? 0 : 1;
    end
endmodule
```

Hazard Detection Unit detects a data hazard in the instruction that occurs when instructions depend on the results of previous instructions still in the pipeline. When a hazard is detected PCwrite is set to 0 which ensures the PC isn't updated. Similarly IFIDWrite is set to 0 which ensures that the IFID register isn't updated and ST is also set to 0 so that all control signals are set to 0 and writeback does not occur and hence introducing a stall in the circuit to delay instruction execution.

14. MUX A

```
`timescale 1ns / 1ps
module MUX_A(
    input [1:0] ForwardA,
    input [31:0] ALUOutEXMEM, WriteData, read_data1IDEX,
    output reg [31:0] ALUin1
    );
    always@(*) begin
    if(ForwardA == 2'b00) begin
        ALUin1 = read_data1IDEX;
    end
    else if(ForwardA == 2'b01) begin
        ALUin1 = WriteData;
    end
    else if(ForwardA == 2'b10) begin
        ALUin1 = ALUOutEXMEM;
    end
    end
endmodule
```

This Mux is placed just before one the input to ALU. It receives the control signal from the data forwarding unit(ForwardA) which decides which data is to be forwarded to ALU. Whn control signal is 0, the data coming from the IDEX register is selected(read_data1IDEX), if control signal is 1, the data is forwarded from the MEMWB pipeline register(WriteData) and if the control signal is 2, the data selected is the one coming from the EXMEM pipeline register.

15. MUX B

```
`timescale 1ns / 1ps
module MUX_B(
    input [1:0] ForwardB,
    input [31:0] ALUin, WriteData, ALUOutEXMEM,
    output reg [31:0] ALUin2
    );
    always@(*) begin
    if(ForwardB == 2'b00) begin
        ALUin2 = ALUin;
    end
    else if(ForwardB == 2'b01) begin
        ALUin2 = WriteData;
    end
    else if(ForwardB == 2'b10) begin
        ALUin2 = ALUOutEXMEM;
    end
    end
endmodule
```

This Mux is placed just before the other input of the ALU, and performs the same function as MUX A.

16. EXMEM PIPELINE REGISTER

```
`timescale 1ns / 1ps
module EXMEM(
    input clk,EnRWIDEX,
    input [31:0] ALUOut,read_data2IDEX,
    input MWIDEX, MRIDEX, MregIDEX,
    input [3:0] WriteRegIDEX,
    output reg [3:0] WriteRegEXMEM,
    output reg [31:0] ALUOutEXMEM, read_data2EXMEM,
    output reg MWEXMEM, MREXMEM, MregEXMEM,EnRWEXMEM
    );
    always@(negedge clk)
    begin
    {EnRWEXMEM, ALUOutEXMEM, WriteRegEXMEM, MWEXMEM, MREXMEM, MregEXMEM} = {EnRWIDEX, ALUOut, WriteRegIDEX, MWIDEX, MRIDEX, MregIDEX};
    read_data2EXMEM = read_data2IDEX;
    end
endmodule
```

This module is used to transfer data and control signals from the Execution (EX) stage to the Memory(MEM) stage. All the input signals and data are latched and then forwarded to the execution cycle. At the negative edge of each clock cycle, the block updates the output registers with the corresponding input values. The size of this register is 72 bits. Since we want the write to happen at the negative edge of the clock, both EXMEM and MEMWB should operate at the negative edge of the clock. Since, for forwarding the contents of both these registers are compared, we want their contents to be updated at the exact same time. If there is a mismatch in that, then we may end up comparing the wrongly updated contents as they are not properly synchronised.

17.DATA MEMORY

```
`timescale 1ns / 1ps
module Data_Memory(
    input MREXMEM, MWEXMEM,
    input[31:0] ALUOutEXMEM,read_data2EXMEM,
    output reg[31:0] Read_data
    );
    reg [7:0]Memory[31:0];
    initial begin
    {Memory[0],Memory[1],Memory[2],Memory[3]} = 32'h00000000;
    {Memory[4],Memory[5],Memory[6],Memory[7]} = 32'h00000000;
    {Memory[8],Memory[9],Memory[10],Memory[11]} = 32'h00000000;
    {Memory[12],Memory[13],Memory[14],Memory[15]} = 32'h00000000;
    {Memory[16],Memory[17],Memory[18],Memory[19]} = 32'h55555555;
    {Memory[20],Memory[21],Memory[22],Memory[23]} = 32'h00000000;
    {Memory[24],Memory[25],Memory[26],Memory[27]} = 32'h00000000;
    {Memory[28],Memory[29],Memory[30],Memory[31]} = 32'h00000000;
    end
    always@(*) begin
    if(MREXMEM)
    begin
    Read_data = {Memory[ALUOutEXMEM], Memory[ALUOutEXMEM + 1], Memory[ALUOutEXMEM + 2], Memory[ALUOutEXMEM + 3]};
    end
    if(MWEXMEM)
    begin
    {Memory[ALUOutEXMEM], Memory[ALUOutEXMEM + 1], Memory[ALUOutEXMEM + 2], Memory[ALUOutEXMEM + 3]} = read_data2EXMEM;
    end
    end
endmodule
```

The Data Memory module stores and retrieves data during program execution. It supports both read and write operations based on control signals MREXMEM and MWEXMEM respectively. When a read control signal is given, the data from the memory is stored into Read_Data from the

address given by the ALUOutEXMEM. When a write control signal is given, the data(read_data2EXMEM) is stored to the address given by ALUOutEXMEM.

18. MEMWB PIPELINE REGISTER

```
`timescale 1ns / 1ps
module MEMWB(
    input clk,EnRWEXMEM,
    input MregEXMEM,
    input [3:0] WriteRegEXMEM,
    input [31:0] Read_data, ALUOutEXMEM,
    output reg MregMEMWB,
    output reg [3:0] WriteRegMEMWB,
    output reg [31:0] ReadDataMEMWB, ALUOutMEMWB,
    output reg EnRWMEMWB
    );
    always@(negedge clk) begin
    {EnRWMEMWB, WriteRegMEMWB,ReadDataMEMWB, ALUOutMEMWB, MregMEMWB} = {EnRWEXMEM, WriteRegEXMEM, Read_data, ALUOutEXMEM, MregEXMEM};
    end
endmodule
```

This module is used to transfer data and control signals from the Memory (MEM) stage to the WriteBack (WB) stage. All the input signals and data are latched and then forwarded to the execution cycle. At the negative edge of each clock cycle, the block updates the output registers with the corresponding input values. The size of this register is 70 bits as the 2 control signals are dropped from the EXMEM stage. This register operates on the negative edge of the clock as well to ensure that we have properly write back the data in the register file.

19.MEMORY TO REGISTER MUX

```
`timescale 1ns / 1ps
module MemtoReg_Mux(
    input MregMEMWB,
    input [31:0] ReadDataMEMWB,ALUOutMEMWB,
    output [31:0] WriteData
    );
    assign WriteData = (MregMEMWB? ReadDataMEMWB : ALUOutMEMWB);
endmodule
```

The mux decides the writedata content of the register file based on the Mreg control signal. The output is ReadData from Data Memory or ALUOut based on the instruction.

20.MAIN MODULE

```
`timescale 1ns / 1ps
module Main_Module(
    input wire clk,
    input wire rst
    );
    wire [1:0]ALUOp,ALUOpIDEX,ALUOp1, ForwardA, ForwardB;
    wire Mreg, zero,zeroEXMEM, EnRW, ALUSrc, MR, MW,EnRWIDEX, ALUSrcIDEX, MRIDEX, MWIDEX, MregIDEX,MregMEMWB;
    wire MWEXMEM, MREXMEM, MregEXMEM, EnIM, EnRWEXMEM, EnRWMEMWB;
    wire EnRW1, ALUSrc1, MR1, MW1, Mreg1, ST,PCWrite, IFIDWrite;
    wire [31:0]PC, MemData, PC_new, Jump_addressEXMEM, MemDataIFID,PC_newIFID, ALUOut,read_data1,read_data2,WriteData,out2, PC_newIDEX, read_data1IDEX, read_data2IDEX, out2IDEX;
    wire [31:0] ALUOutEXMEM,read_data2EXMEM, Read_data, ALUin,ALUOutMEMWB, ALUin1, ALUin2,ReadDataMEMWB;
    wire [3:0]WriteRegIDEX, WriteReg, WriteRegEXMEM, WriteRegMEMWB, read_reg1, read_reg2, read_reg1IDEX, read_reg2IDEX;
    PC_Register P1(PCWrite,clk,rst,PC_new,PC);
    Instruction_Memory M1(EnIM,PC, MemData);
    PC_Adder P12(PC,PC_new);
    IFID I1(clk, IFIDWrite, MemData, MemDataIFID);
    Control_Signal CS1(MemDataIFID,EnRW1, ALUSrc1, MR1, MW1, Mreg1, ALUOp1);
    Hazard_detection HD12(MRIDEX,MemDataIFID,WriteRegIDEX,IFIDWrite,PCWrite, ST, EnIM);
    Register_File R112(EnRWMEMWB,MemDataIFID, WriteData, WriteRegMEMWB,read_data1, read_data2, WriteReg, read_reg1, read_reg2);
    Control_Mux CM12(EnRW1, ALUSrc1, MR1, MW1, Mreg1,ALUOp1,ST,EnRW, ALUSrc, MR, MW, Mreg,ALUOp);
    Sign_Extend S123(MemDataIFID,out2);
    IDEX I9(read_reg1, read_reg2, clk, read_data1, read_data2, out2,WriteReg,EnRW, ALUSrc, MR, MW, Mreg,ALUOp,read_data1IDEX, read_data2IDEX, out2IDEX,WriteRegIDEX
    ,EnRWIDEX, ALUSrcIDEX, MRIDEX, MWIDEX,MregIDEX,ALUOpIDEX, read_reg1IDEX, read_reg2IDEX);
    ALUSrc_Mux A12454(ALUSrcIDEX,read_data2IDEX, out2IDEX,ALUin);
    Forwarding_Table FT11(WriteRegEXMEM, WriteRegMEMWB,read_reg1IDEX, read_reg2IDEX,ForwardA, ForwardB);
    MUX_A MUXA1(ForwardA,ALUOutEXMEM, WriteData, read_data1IDEX,ALUin1);
    MUX_B MUXB123(ForwardB,ALUin, WriteData, ALUOutEXMEM,ALUin2);
    ALU AL1(ALUOpIDEX, ALUin1, ALUin2, ALUOut);
    EXMEM E123(clk,EnRWIDEX, ALUOut,read_data2IDEX, MWIDEX, MRIDEX, MregIDEX,WriteRegIDEX,WriteRegEXMEM,ALUOutEXMEM, read_data2EXMEM, MWEXMEM, MREXMEM, MregEXMEM, EnRWEXMEM);
    Data_Memory D11(MREXMEM, MWEXMEM,ALUOutEXMEM,read_data2EXMEM,Read_data);
    MEMWB MEM1(clk,EnRWEXMEM,MregEXMEM,WriteRegEXMEM,Read_data, ALUOutEXMEM,MregMEMWB,WriteRegMEMWB,ReadDataMEMWB, ALUOutMEMWB,EnRWMEMWB);
    MemtoReg_Mux MR12(MregMEMWB,ReadDataMEMWB,ALUOutMEMWB,WriteData);
endmodule
```

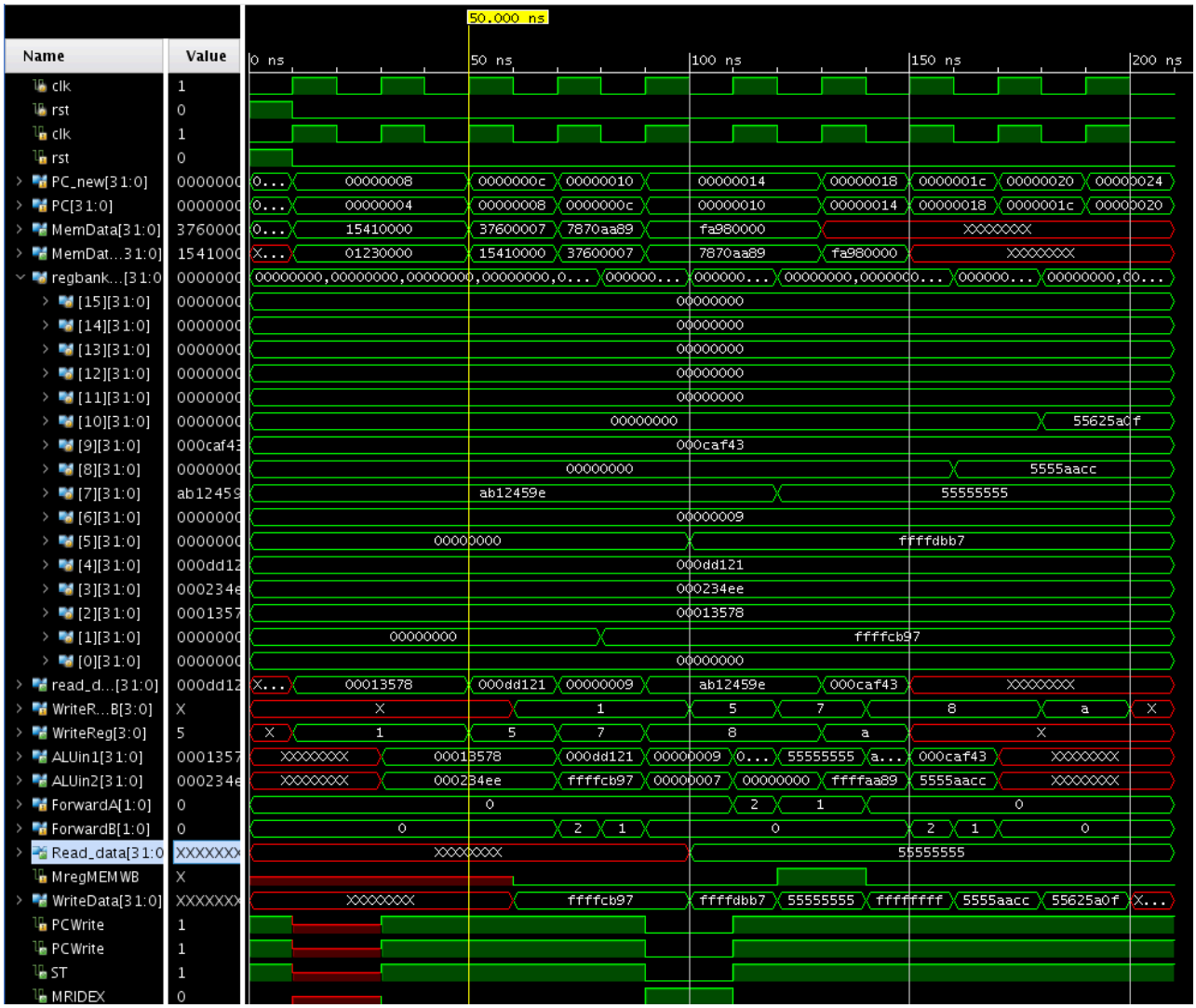
The main module is where we have called all the individual components in the order of the datapath, i.e. in the order they will be executed. We have also ensured that the naming of the variables stays constant to what it is in all the individual modules.

21.TESTBENCH

```
`timescale 1ns / 1ps



module tb();
reg clk,rst;
   Main_Module m1(.clk(clk),.rst(rst));

   initial begin
   rst=1;
   #10 rst=0;
   #200 $stop;
   end

   initial begin
   clk=0;
   forever
   #10 clk=~clk;
   end


endmodule
```

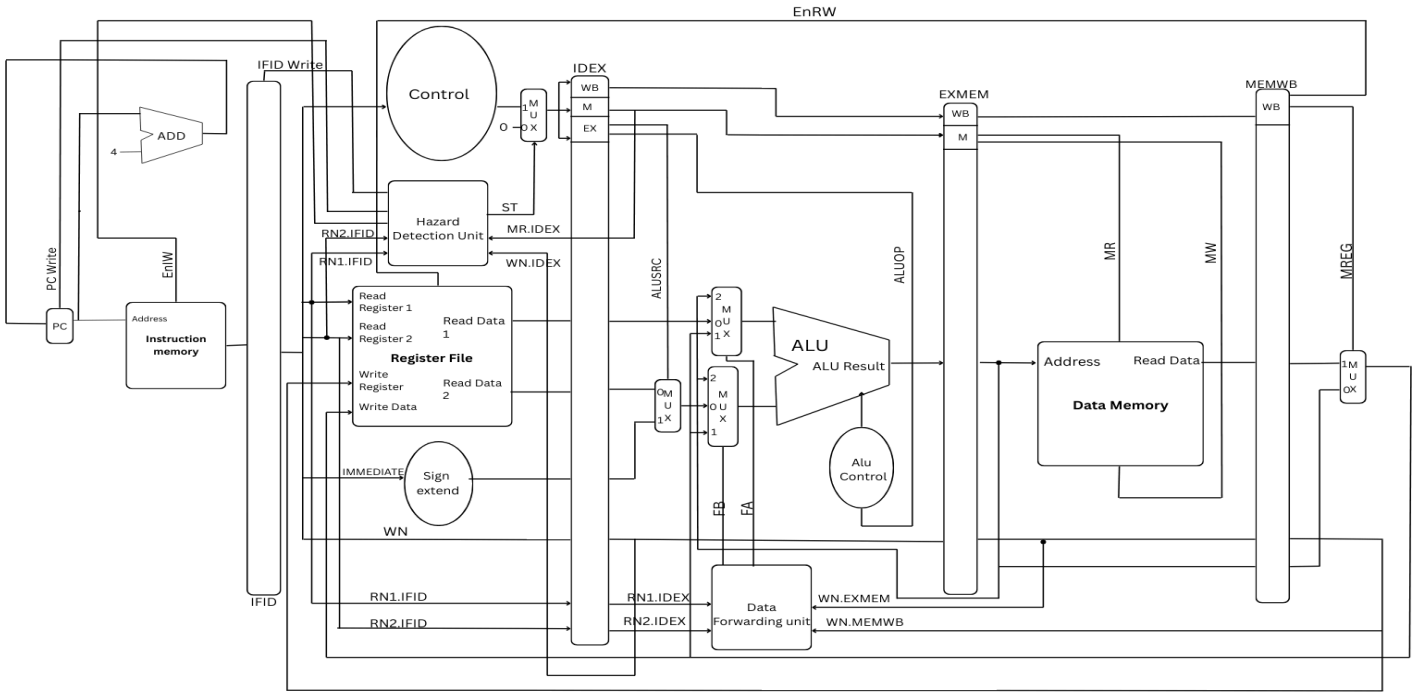
The testbench only consists of clock and reset which are given to the main module, from where they are given to each of the pipeline registers.

22.WAVEFORM



As can be seen from the waveform, the contents are being updated perfectly. Exactly when the load instruction is detected to be having a hazard, the PCWrite, ST values become 0 and the MRIDEX becomes 1, thus introducing a stall. All the write operations are happening at the negative edge of the clock and the read operations at the positive edge, which can be seen in this figure. The variation of many other parameters with the changing clock, justifying why the results are obtained as they are can also be seen in the waveform.

BLOCK DIAGRAM:



CONCLUSION

Upon synthesizing our project with all these modules, we got the above block diagram, which is functioning properly according to the set of instructions provided to us. This project will run even when the instructions are arranged in any different order. We were also able to implement this processor such that the reading operation took place at the positive edge of the clock and the writing operation took place at the negative edge of the clock as specified in the project, using a minimum amount of hardware.

CONTRIBUTIONS:

All of us have contributed equally in the completion of this project.